

Add-Migration vs Update-Database

Add-Migration vs Update-Database — Explained

Why two separate steps exist

Reasoning: EF Core never touches your real database automatically. When you change a model (say, add a new property to Student), EF doesn't know yet *how* that change should turn into real SQL. So the process is split into two stages:

1. **Write down the plan** — “what changed, and what SQL would apply it” → this is Add-Migration
2. **Actually run the plan** — “execute that SQL against the real database” → this is Update-Database

Think of it like writing a recipe first, then actually cooking it. You can write several recipes before you decide to cook any of them.

Step 1 — Add-Migration (or `dotnet ef migrations add`)

What it actually does:

- EF Core compares your current models (Student, Courses, etc.) against the *last known snapshot* it has saved (a hidden file it keeps track of).
- It figures out the difference — e.g., “a new column StudentEmail was added to Student.”
- It generates a new **migration file** in your Migrations folder — a C# file with two methods:
 - `Up()` — the code to apply the change (e.g., add the column)
 - `Down()` — the code to undo the change (e.g., remove the column), in case you need to roll back later

Important: at this point, **nothing has touched your real database yet**. It's purely a file being created on your computer.

```
Add-Migration AddStudentEmail
```

or

```
dotnet ef migrations add AddStudentEmail
```

AddStudentEmail here is just a name you choose — like naming a recipe so you remember what it's for later.

Step 2 — Update-Database (or `dotnet ef database update`)

What it actually does:

- EF Core looks at all migrations that haven't been applied yet (including the one you just created).
- It runs their `Up()` code against your actual SQL Server database — this is the moment real tables/columns actually get created or changed.
- EF also keeps a special table inside your database (`__EFMigrationsHistory`) to remember which migrations have already been applied, so it never re-applies the same one twice.

`Update-Database`

or

`dotnet ef database update`

The Key Difference, Summed Up

	Add-Migration	Update-Database
What it touches	Only files on your computer (Migrations folder)	The real database
When to use it	Every time you change a model	Whenever you're ready to apply pending migrations
Can you undo it easily?	Yes — just delete the migration file if unapplied	Harder — you'd need another migration or a rollback command
Analogy	Writing the recipe	Actually cooking it

A Concrete Example, Step by Step

1. You add `public string StudentEmail { get; set; }` to your `Student` class.
2. Run `Add-Migration AddStudentEmail` → EF creates a migration file describing "add column StudentEmail."
3. Run `Update-Database` → EF actually adds that column to `tblStudents` in SQL Server.

If you skip step 3, your code will compile fine, but your database table still won't have the new column — and your app will crash the moment it tries to read/write `StudentEmail`, because the database doesn't know about it yet.